

Katholieke
Universiteit
Leuven

**Faculty of
Engineering Technology
Ghent**

GEAVANCEERDE COMPUTERARCHITECTUUR

Labo 1

Tano Pannekoucke
Academic year 2025–2026

Inhoudsopgave

1 Context **3**

2 Methodology **3**

3 Observations/Results **3**

4 Conclusion **4**

5 References **4**

1 Context

In deze eerste CUDA-oefening werd een eenvoudige toepassing ontwikkeld die een array omkeert op de GPU. Het doel is om inzicht te verwerven in het geheugenbeheer tussen **host** (CPU) en **device** (GPU) en om te tonen hoe threads en blocks samenwerken. De implementatie gebruikt een kernel `flip` die elk element op index i in de inputarray naar positie $n-1-i$ in de outputarray schrijft.

2 Methodology

Gegevensoverdracht: host → device → host

De host maakt twee arrays aan, `a` en `b`, in het CPU-geheugen. Vervolgens wordt via `cudaMalloc()` geheugen gereserveerd op de GPU (`d_a` en `d_b`). De data uit `a` wordt naar de device gekopieerd met:

```
cudaMemcpy(d_a, a, N * sizeof(int), cudaMemcpyHostToDevice);
```

Na de berekening op de GPU wordt het resultaat met `cudaMemcpy()` in omgekeerde richting (`cudaMemcpyDeviceToHost`) teruggestuurd naar de host. Zo gebeurt de berekening volledig op de GPU, terwijl de CPU instaat voor initialisatie en het ophalen van de resultaten.

CUDA hanteert een hiërarchie: threads → blocks → grid.

- `threadIdx`: lokale coördinaten van de thread binnen het block; een 3D-structuur (`.x`, `.y`, `.z`). In 1D gebruiken we enkel `.x`, met bereik $0 \leq$ `threadIdx.x` $<$ `blockDim.x`.
- `blockIdx`: coördinaten van het block binnen het grid (welk block voer ik uit?).
- `blockDim`: afmetingen (aantal threads) van een block, constant voor alle threads in dat block.

Een **globale** 1D-index volgt uit:

```
int idx = threadIdx.x + blockIdx.x * blockDim.x;
```

waar `threadIdx.x` de lokale positie in het block is van de thread en `blockIdx.x` × `blockDim.x` de offset van het block in het grid. Samen levert dit een unieke index `idx` op voor het mappen van threads naar data-elementen.

Meerdere blocks en threads per block

De kernel wordt gelanceerd met een configuratie die meerdere blocks toelaat:

```
const int TPB   = 256;                      // Threads per block
const int blocks = (N + TPB - 1) / TPB;     // Aantal blocks
flip<<<blocks, TPB>>>(d_a, d_b, N);
```

De globale index van een thread wordt in de kernel berekend met:

```
int idx = threadIdx.x + blockIdx.x * blockDim.x;
```

Hierdoor kan elke thread één element verwerken; de *bounds-check* `if (idx < n)` vermijdt out-of-bounds toegang wanneer N niet deelbaar is door `TPB`. Het gebruik van meerdere blocks verdeelt het werk over meerdere Streaming Multiprocessors (SM's), wat de schaalbaarheid en bezetting verbetert.

3 Observations/Results

Voor de test `N` $= 10$ en `a` $= [0, 1, 2, \ldots, 9]$ resulteert de uitvoer in `b` $= [9, 8, 7, \ldots, 0]$, wat de correctheid van de flip aantoont. De gekozen launch-configuratie is schaalbaar: bij grotere `N` zorgt `blocks` $= \lceil N/$`TPB`$\rceil$ ervoor dat voldoende blocks worden voorzien. Opeenvolgende threads lezen `a[idx]` in stijgende volgorde en schrijven `b[n-1-idx]` in dalende, maar aaneensluitende adressen per warp, wat in de praktijk efficiënte globale geheugenaccessen ondersteunt.

4 Conclusion

Deze oefening illustreert hoe data van de CPU (host) naar de GPU (device) wordt overgebracht en hoe een eenvoudige, maar representatieve, *data-parallel* kernel over meerdere blocks en threads kan worden opgeschaald. De gekozen configuratie (`TPB` = 256) voldoet aan de richtlijn "*meerdere blocks met meerdere threads*" en benut de GPU-hardware beter dan een enkel block, zeker voor grotere inputgroottes.

Maximale blockgrootte

Volgens de CUDA Programming Guide bedraagt de **maximale blockgrootte 1024 threads per block**. Deze limiet is een gevolg van de architectuur van de Streaming Multiprocessor (SM): alle threads binnen één block delen middelen zoals registers en *shared memory*. Het getal 1024 correspondeert met **32 warps** van telkens **32 threads** (warp size), wat een goede balans biedt tussen rekenkracht, scheduling en geheugenverbruik.

5 References

- NVIDIA, *CUDA C++ Programming Guide*, NVIDIA Developer Documentation.
- KU Leuven, *Geavanceerde Computerarchitectuur – Session 1*, oefening "Flip Array".

```cpp
#include <iostream>
#include <cuda_runtime.h>

// CUDA kernel flip array
__global__ void flip(int *a, int *b, int n) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x; // Calculate global thread index. threadIdx.x is the index within the block, blockIdx.x is the block index and blockDim.x is the number of threads per block.

    if (idx < n) {
        b[n - idx -1] = a[idx];
    }
}

int main() {
    const int N = 10;
    const int TPB = 256; // Threads per block
    int blocks = (N + TPB - 1) / TPB;

    int a[N], b[N];
    int *d_a = nullptr, *d_b = nullptr;

    // Initialize host arrays
    for (int i = 0; i < N; i++) {
        a[i] = i;
    }

    // Allocate device memory
    cudaMalloc((void**)&d_a, N * sizeof(int));
    cudaMalloc((void**)&d_b, N * sizeof(int));

    // Copy data from host to device
    cudaMemcpy(d_a, a, N * sizeof(int), cudaMemcpyHostToDevice);

    // Launch kernel met meerdere blocks
    flip<<<blocks, TPB>>>(d_a, d_b, N);
    cudaDeviceSynchronize();

    // Copy result from device back to host
    cudaMemcpy(b, d_b, N * sizeof(int), cudaMemcpyDeviceToHost);

    // Print results
    std::cout << "Result: ";
    for (int i = 0; i < N; i++) {
        std::cout << b[i] << " ";
    }
    std::cout << std::endl;

    // Free device memory
    cudaFree(d_a);
    cudaFree(d_b);

    return 0;
}
```

Listing 1: CUDA kernel om een array te flippen